

# kyagent 软件功能设计文档

赛题：A2「面向麒麟操作系统的安全智能运维 Agent」 作品：kyagent 目标平台：LoongArch64  
Linux + 麒麟高级服务器版 V11 文档版本：v1.0

## 1. 设计目标与原则

本文档对应赛题提交要求第 2 项「软件功能设计文档」，描述 kyagent 的总体架构、模块设计、关键数据流、安全模型、配置与权限设计、可扩展性设计。所有设计均引用真实源码路径与类/函数。

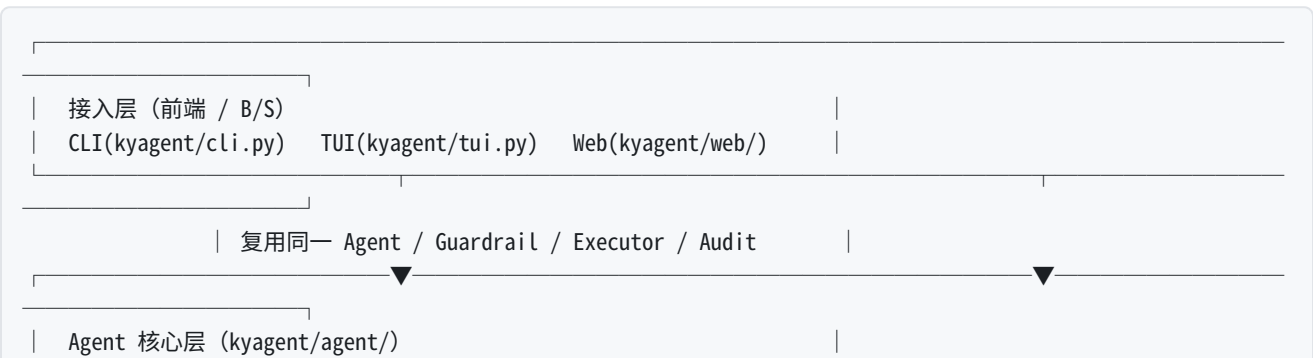
设计遵循三条核心原则：

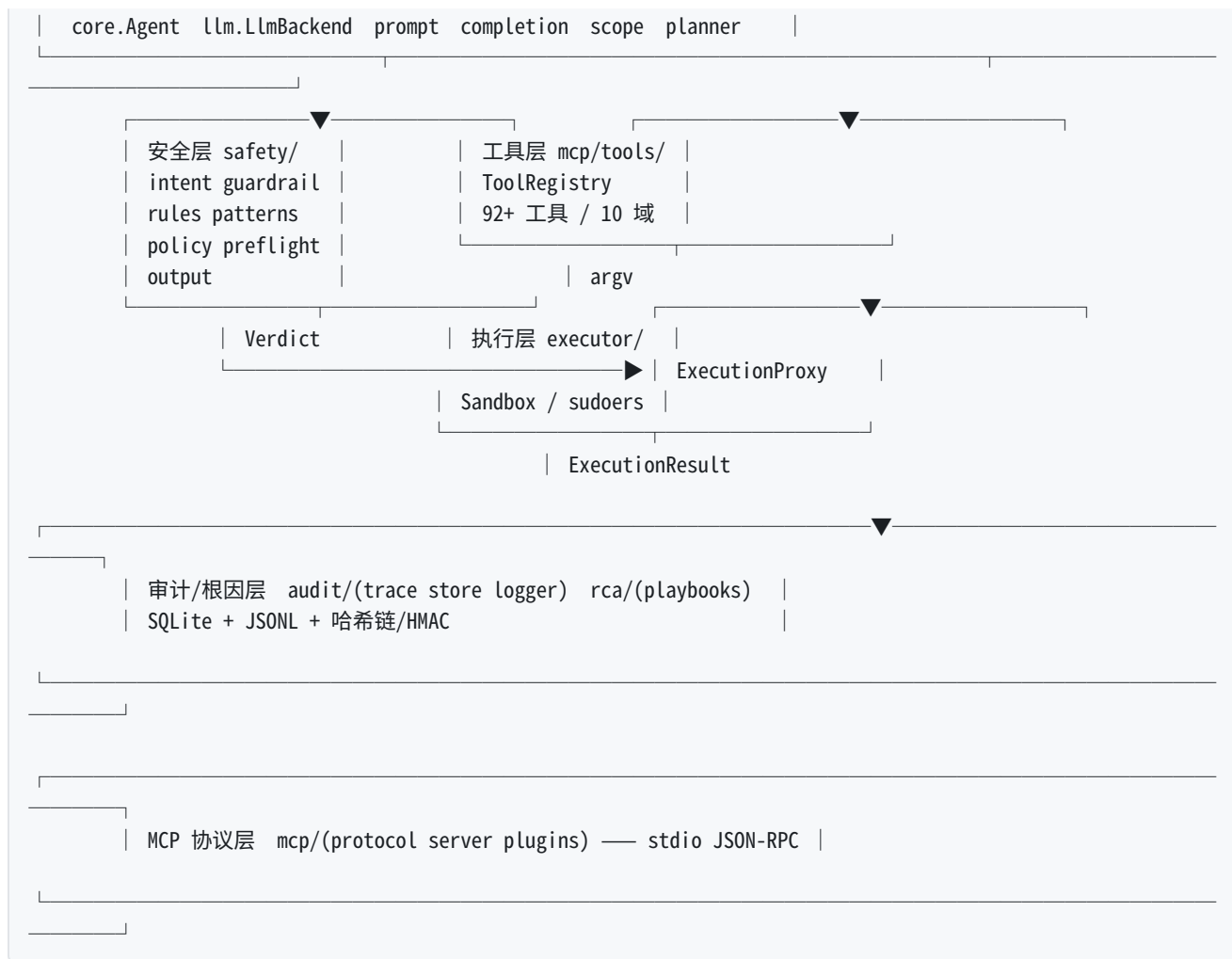
1. 受控模板，而非自由 shell：LLM 只能选择工具与参数；工具自己把参数转成 argv 列表，不执行 LLM 拼出来的 shell 字符串，从源头降低注入风险。
2. 分层防御：安全不是单层，而是 IntentGuard → Tool.validate → build\_argv → Guardrail → Confirm（人在回路）→ ExecutionProxy → sudoers → Audit 多层叠加，单层失效不会直接放行危险操作。
3. 单向依赖、数据结构通信：模块之间靠 Verdict / ExecutionResult / ToolResult 等数据结构通信，不互相 import 内部实现，便于替换与测试。

原则 1 在代码中的三处落点：① 工具用代码构造 argv 而非拼接字符串—— kyagent/mcp/tools/base.py 的 Tool.build\_argv() 返回 list[str]，并拒绝 shell 元字符；| & \$ \ 换行；② 执行层不使用 shell=True —— kyagent/executor/proxy.py 的 subprocess.Popen(final\_argv, ...) 直接以 argv 列表 execve；③ 系统层兜底—— configs/sudoers.kyagent 仅放行绝对路径白名单命令，并拒绝 /bin/sh /bin/bash /usr/bin/python\* perl awk sed` 等解释器。三者叠加，使 LLM 无论如何都无法获得一个可拼接的 shell。

## 2. 总体架构

### 2.1 分层视图





## 2.2 模块边界契约（引自 [docs/kyagent/architecture.md](https://docs.kyagent.com/architecture.md)）

| 模块                              | 状态   | 边界契约                                                                                                        |
|---------------------------------|------|-------------------------------------------------------------------------------------------------------------|
| <code>kyagent.agent.Llm</code>  | 无状态  | 仅做 <code>chat(system, messages, tools) → AssistantMessage</code> ，不感知工具语义                                   |
| <code>kyagent.agent.core</code> | 有上下文 | 持有 <code>messages</code> 与子系统句柄，负责 <code>trace</code> 串联与多轮迭代                                               |
| <code>kyagent.mcp.tools</code>  | 无状态  | <code>Tool.build_argv</code> 只产 <code>argv</code> ，不真正执行、不做安全决策                                             |
| <code>kyagent.safety</code>     | 无状态  | 输入 <code>cmdline/argv + declared_risk</code> ，输出 <code>Verdict</code>                                       |
| <code>kyagent.executor</code>   | 无状态  | 接受 <code>argv + requires_root</code> ，副作用仅限于进程                                                              |
| <code>kyagent.audit</code>      | 持久化  | 唯一向 <code>SQLite/JSONL</code> 写入的入口                                                                         |
| <code>kyagent.mcp.server</code> | 进程   | <code>stdio JSON-RPC</code> ，把模块组合成 <code>MCP host</code> 可见工具                                              |
| <code>kyagent.web.server</code> | 进程   | <code>FastAPI B/S</code> 接入层， <code>ProgressEvent → SSE</code> ，浏览器 <code>approve/reject → ConfirmFn</code> |

## 3. 各模块设计

### 3.1 Agent 核心（`kyagent/agent/core.py`）

Agent 是有上下文的编排器：持有 `messages`、`LlmBackend`、`ToolRegistry`、`Guardrail`、`ExecutionProxy`、`AuditLogger` 句柄。核心方法 `Agent.ask()`：

1. 开 trace (`uuid`) → `AuditLogger.open`，落 `USER_INPUT`。
2. `IntentGuard` 预过滤 → 落 `INTENT_CHECK`。
3. 进入"LLM ↔ 工具"多轮循环（`max_iterations` 控制，默认见 `configs/default.yaml`）：
  - `LlmBackend.chat()` 返回 `AssistantMessage(blocks=[text, tool_use])`；- 纯文本 → `AGENT_REPLY`；- `tool_use` → `Tool.validate + build_argv` → `Guardrail.check_argv` → (`allow / confirm+同意`) → `ExecutionProxy.run` → `Tool.format_result` → 把 `tool_result` 追加回 `messages`，继续迭代直到 `stop_reason=end_turn`。
4. 关闭 trace → `AuditLogger.close`。

流式与并发：`Agent.ask()` 额外暴露 `on_progress`（事件 kind：`agent_start/thinking_start/thinking_delta/thinking_end/tool_call_start/tool_call_end/user_choice/agent_final/error`，schema 见 `kyagent/progress.py`）与 `on_user_choice`（驱动 `ask_user_choice` 工具，schema 见 `kyagent/interactive.py`）。

`Agent._active_run_thread_id` 区分 turn 拥有线程与并行 tool worker：只有拥有线程能发起 `confirm`，并行 worker 不得触发 `confirm`（见 `tests/test_agent_parallel.py`）。

LLM 后端抽象（`kyagent/agent/llm.py`）：`LlmBackend.chat()` 统一返回 `AssistantMessage`。支持后端名：`mock / openai / deepseek / qwen / openai_httpx / deepseek_httpx / qwen_httpx / anthropic`。新增 `chat_stream(system, messages, tools, on_delta)`：基类提供基于 `chat()` 的 fallback，`HttpxBckend`（`OpenAI SSE`）、`OpenAIBackend`、`MockBackend` 各自原生实现；`AnthropicBackend` 走 fallback（避免 `jiter Rust` 编译，对 `LoongArch` 不友好）。

`LoongArch/Kylin` 默认推荐 `deepseek_httpx`：只依赖 `httpx`，避开 `jiter`、`pydantic-core` 等在 `LoongArch Old World` 难装的依赖。key 仅通过环境变量注入（如 `DEEPSEEK_API_KEY`），`configs/deepseek.yaml` 的 `api_key_env` 存的是变量名而非 key 本体。

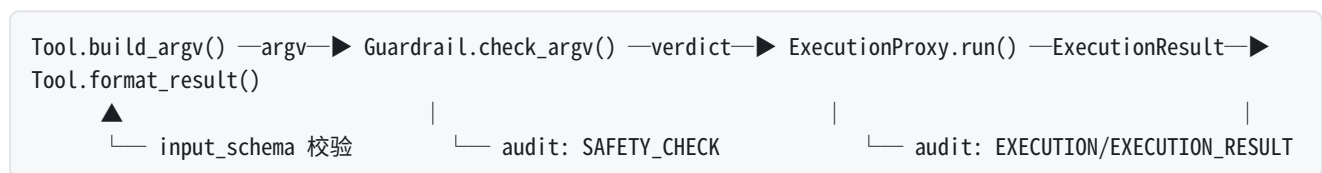
Prompt / Completion / Scope：`prompt.py` 构造系统提示与工具引导；`completion.py` 处理多轮收敛；`scope.py` 控制暴露给 LLM 的工具范围（与 `mcp/plugins.py allowlist` 协同）。`planner.py + mcp/tools/planstate.py` 提供可持久化的任务计划（plan DB）。

### 3.2 MCP 协议与工具插件（`kyagent/mcp/`）

- 协议层（`protocol.py`、`server.py`）：自研 MCP stdio JSON-RPC server，`kyagent mcp serve` 即标准 stdio，可被 `Claude Desktop / Cursor / Continue` 直接发现，不依赖官方 `mcp SDK`（避开 `LoongArch Old World` 依赖风险）。
- 插件 allowlist（`plugins.py`）：显式控制启用的工具域。
- 工具基类（`tools/base.py`）：`Tool` 声明 `name / description / input_schema / risk_level`，并实现 `validate / build_argv / format_result`。两个基础设施类约束实现边界：

- `TrendTool` (`base.py`) —— 采样型工具基类，约定把"间隔/次数"编入命令本身（如 `iostat -dx 1 2`），绝不在 Python 端 `sleep`，避免给 sudoers 放行 `sh -c "...; sleep N"` 这类拼接命令。子类目前为 `DiskIoStatsTool`。
- `PkgFamilyMixin` (`pkg_family.py`) —— 一次性读 `/etc/os-release` 判定 `PkgFamily.RPM`（麒麟服务器版/RHEL 系）或 `DPKG`（麒麟桌面版/Debian 系），进程级缓存；`PkgVerifyTool` 在 RPM 下发 `rpm -V`、DPKG 下发 `debsums -c`，LLM 只看到统一的 `pkg_verify`。
- 工具注册 (`tools/__init__.py`)：`register_builtin()` 按域注册 `process/network/logs/service/filesystem/package/disk/system/security/compliance/loongarch` 等基础感知域，以及 `gitinspect/validation/webfacts/docintake/planstate/runtime_state/permissions/cron/interactive` 扩展域；`default_registry()` 一次性串起。

工具与三大子系统是严格单向依赖：



唯一例外是 `interactive.AskUserChoiceTool`：由 `Agent._handle_tool_use_inner` 按名拦截，hand-off 给 UI 回调而不走 `ExecutionProxy`，`build_argv` 只返回 placeholder。

主要工具域（节选自 `docs/kyagent/README.md`）：

| 工具域        | 代表工具                                                                                                                                                                                                                                              |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| process    | <code>process_list</code> <code>lsof_port</code> <code>lsof_pid</code> <code>process_zombies</code> <code>process_tree</code> <code>process_fd_count</code> <code>process_resource</code> <code>top_cpu_snapshot</code> <code>process_kill</code> |
| network    | <code>net_listen</code> <code>net_connections</code> <code>net_routes</code> <code>net_arp</code> <code>net_addr</code> <code>net_dns_resolve</code> <code>net_firewall_iptables</code> <code>net_firewall_nft</code> <code>net_tcp_stats</code>  |
| logs       | <code>log_journal</code> <code>log_dmesg</code> <code>log_files_top</code> <code>log_size_sample</code> <code>log_grep_recent</code> <code>log_auth_failed</code> <code>log_vacuum</code> <code>log_delete_file</code>                            |
| service    | systemd 状态/列表/ reload / restart （受控变更）                                                                                                                                                                                                            |
| filesystem | <code>fs_df</code> <code>fs_du</code> <code>fs_ls</code> <code>fs_find</code> <code>file_cleanup_candidates</code> <code>fs_truncate</code> <code>fs_delete_file</code>                                                                           |
| package    | <code>pkg_info</code> <code>pkg_installed</code> <code>pkg_verify</code> <code>pkg_updates</code> <code>pkg_install</code> <code>pkg_update</code> <code>pkg_remove</code> （写操作需确认）                                                               |
| disk       | <code>disk_io_stats</code> <code>disk_io_diskstats</code> <code>disk_inode_usage</code> <code>disk_open_deleted</code> <code>disk_mount</code> <code>disk_smart</code> <code>dir_largest_files</code>                                             |
| system     | <code>sys_uptime</code> <code>sys_loadavg</code> <code>sys_memory</code> <code>sys_kernel</code> <code>sys_cpu_info</code> <code>sys_dmi</code> <code>sys_time_sync</code> <code>sys_block_devices</code>                                         |
| security   | <code>sec_selinux_status</code> <code>sec_apparmor_status</code> <code>sec_kysec_status</code> <code>sec_setuid_files</code> <code>sec_world_writable</code> <code>sec_sudoers_audit</code> <code>sec_listening_external</code>                   |
| compliance | <code>compl_aide_check</code> <code>compl_file_hash</code> <code>compl_file_attr</code> <code>compl_timestamp_audit</code> <code>compl_cron_dump</code>                                                                                           |

| 工具域           | 代表工具                                                                        |
|---------------|-----------------------------------------------------------------------------|
| loongarch     | la_arch_info la_world_check la_binary_compat                                |
| runtime_state | lock_inspect lock_remove_stale unix_socket_inspect unix_socket_remove_stale |
| cron          | cron_d_list cron_d_read cron_entry_trace cron_d_disable                     |
| permissions   | log_dir_repair_permissions                                                  |
| interactive   | ask_user_choice submit_rca_report                                           |

### 3.3 安全层（`kyagent/safety/`）

| 文件                     | 职责                                                                                       |
|------------------------|------------------------------------------------------------------------------------------|
| intent.py              | IntentGuard：用户原始自然语言进入 LLM 前的意图风险预过滤 + prompt injection 识别                               |
| guardrail.py           | Guardrail.check_argv()：对最终 argv 做规则裁决，工具 declared_risk 作为下限，可选 LLM 复审（只升级不降级），输出 Verdict |
| rules.py / patterns.py | 规则引擎与正则集合，RuleEngine.from_yaml(configs/safety-rules.yaml)                                |
| policy.py              | risk → decision 策略映射                                                                     |
| write_preflight.py     | 写操作前置校验：路径归一化（ <code>posixpath.normpath</code> ）、越界/符号链接防护                               |
| output.py              | 输出脱敏，防止密钥/敏感信息回流                                                                         |

规则库三类规则（`configs/safety-rules.yaml`）：

1. 正则规则（`pattern`）：针对字符串形态的危险写法，例：`pipe-to-shell` 命中 `(curl|wget|fetch)...|(bash|sh|...)`。
2. argv 规则（`command + flags + target`）：针对意图明确的命令，例：`rm-recursive-system`（`command=rm`，`flags_any=[-rf,-fr,-r,-R,--recursive]`，`target_in=[/etc,/usr,/var,...]`）。规则内字段 AND，规则间 OR。
3. 等价 flag 拆分：`-rf` 自动展开为 `{-r,-f}`，与 `flags_all` 匹配，避免 `-r -f / -rf / -fr` 绕过。

risk → decision 默认策略（`configs/default.yaml`）：

| risk     | decision |
|----------|----------|
| critical | deny     |
| high     | confirm  |
| medium   | confirm  |
| low      | allow    |

declared\_risk 下限机制: `svc_restart.risk_level = HIGH` 且 `requires_root = True` , 即便参数完全合法 ( `systemctl restart nginx` ) , Guardrail 也会把 risk 升到 HIGH → CONFIRM, 不依赖正则, 是"工具内禀风险"兜底。

### 3.4 执行层 ( `kyagent/executor/` )

`ExecutionProxy.run()` ( `proxy.py` ) + `Sandbox` ( `sandbox.py` ) :

- PATH 白名单解析, `sudo -n -u <target> --` 包裹 (target 默认 `kyagent` ) ;
- `subprocess.Popen` 列表参数 (无 `shell=True` ) ;
- 干净 env (去 `LD_PRELOAD` 等) ;
- rlimit CPU/AS/FSIZE/NOFILE;
- 独立进程组 + timeout (SIGTERM/SIGKILL pgrp) ;
- stdout/stderr cap (sandbox `output_cap` 64K/流 + truncated 标记; 工具层再统一截到 6KB, 见 `pipeline.OUTPUT_CAP` ) 。

`forbid_root=true` 含义是"非必要不用 root; 声明 `requires_root=True` 的工具交给 sudoers 判断";  
`forbid_root_strict=true` 才彻底禁用 root 提权。

### 3.5 审计/溯源层 ( `kyagent/audit/` )

- `trace.py` : `Trace` (唯一 `trace_id` 串联事件)、`TraceEvent` (含 `prev_hash` / `event_hash` / `event_hmac` / `key_id` )、`EventKind` 枚举: `USER_INPUT` / `INTENT_CHECK` / `PERCEPTION` / `DIAGNOSIS` / `LLM_THOUGHT` / `TOOL_REQUEST` / `SAFETY_CHECK` / `EXECUTION` / `EXECUTION_RESULT` / `PLAN_UPDATE` / `BUDGET` / `AGENT_REPLY` / `ERROR` 。
- `store.py` : 唯一向 SQLite 写入的入口; `logger.py` : `AuditLogger.open/event/close` , 双写 SQLite + JSONL。
- 只读工具结果落为 `PERCEPTION evidence_id` ; 生产可启用 HMAC 封印,  
`kyagent audit verify <trace-id>` 校验链完整性。JSONL 可外送 SIEM, 攻击者无法只删本机。

### 3.6 根因分析层 ( `kyagent/rca/` )

- `playbooks.py` + `configs/rca-playbooks.yaml` : 内置 `zombie_process` / `disk_io` / `config_drift` / `large_logs` 四个 playbook, 每个声明 `description` / `evidence` / `checks` 。
- `report.py` : 结构化 RCA 报告生成; `submit_rca_report` 工具要求选择内置 playbook, 并只能引用当前 trace 中已存在的 evidence, 避免凭空编造根因。

### 3.7 前端层 (CLI / TUI / Web)

- CLI ( `kyagent/cli.py` ) : `kyagent ask/chat/tools/safety/audit/tui/web/mcp` 等子命令。 `ask` 单轮模式对 confirm 默认 `n` 。
- TUI ( `kyagent/tui.py` ) : `prompt_toolkit` + `rich` , 命令 `/tools` / `/audit` / `/reset` / `/exit` , 不引入 `Textual` / `tree-sitter` , 只负责展示与确认, 复用同一 `Agent.from_config()` 。
- Web ( `kyagent/web/` ) : FastAPI + 同源静态前端 (零外部 Node 进程) 。 `server.py` 把 `ProgressEvent` 推成 SSE; `approvals.py` 的 `ApprovalBroker` 在遇到 `ConfirmRequest` 时创建待审核记录并阻塞当前

turn, 浏览器 approve/reject 唤醒执行线程; security.py 负责非回环监听的认证 (四类 token, fail closed); schemas.py / choices.py 定义请求与选择结构。

## 4. 关键数据流 (指令 → 感知 → 推理 → 安全 → 执行 → 审计闭环)

```
user_input "重启 nginx"
  | Agent.ask() 开 trace → AuditLogger.open / USER_INPUT
  ▼ IntentGuard → INTENT_CHECK
LlmBackend.chat() (带 tools schema)
  ▼ AssistantMessage(blocks=[text, tool_use])
text only → AGENT_REPLY
tool_use → Tool.validate + build_argv (禁 shell 元字符) → argv / TOOL_REQUEST
  ▼ Guardrail.check_argv() declared_risk=tool.risk_level → Verdict / SAFETY_CHECK
  |— allow —————→
  |— confirm — confirm_cb() — true
  |   |— false —→ ToolResult(is_error, "[denied]...")
  |— deny —————→ ToolResult(is_error, "[denied]...")
  ▼ (true)
ExecutionProxy.run() sudo -n -u kyagent -- timeout/rlimit/output_cap/PATH 白名单
  ▼ ExecutionResult / EXECUTION + EXECUTION_RESULT
Tool.format_result() → ToolResultBlock → messages.append(tool_result) → PERCEPTION
  |— 回 LlmBackend.chat() 多轮迭代 直到 stop_reason=end_turn
  ▼ AGENT_REPLY → AuditLogger.close
```

每一步都有审计落点, 构成赛题要求的"接收指令 → 感知环境 → 推理决策 → 安全校验 → 执行结果"闭环。

## 5. 安全模型设计

### 5.1 威胁模型

把 LLM 视为"半可信组件", 可能: 幻觉危险工具调用、越权读写关键文件、被注入劫持、意外生成有效但破坏性的命令。假设 OS、sudoers 配置受信, 审计存储未被同环境攻击者篡改。

### 5.2 防御分层 (docs/kyagent/safety-model.md)

安全检查分布在 7 个层次, 各层职责单一、互不依赖对方实现; 单层失效不会直接导致危险操作落地:

```
(赛题前置) IntentGuard 自然语言意图/注入预过滤 — 不可信输入进 LLM 前先筛一道
LLM 输出工具调用 (只能是结构化 tool_use, 不是 shell 字符串)
  |
  ▼ [L1] Tool.validate() JSON Schema required/type/enum/pattern 校验, 拒绝未知字段
  ▼ [L2] Tool.build_argv() 禁 shell 元字符; 限定子命令枚举; 路径黑名单; 代码构造 argv 列表
  ▼ [L3] Guardrail.check() 规则引擎; declared_risk 下限; 可选 LLM 复审 (只升级不降级); risk→decision
  ▼ [L4] Confirm (人在回路) high/medium 需人工确认: CLI y/N、Web 审批卡片; 非交互/并行 worker 默认拒绝
  ▼ [L5] ExecutionProxy.run() PATH 白名单; sudo -n -u; 干净 env (去 LD_PRELOAD); rlimit; 进程组 timeout; 输出 cap
  ▼ [L6] sudoers 系统层最终把关, 命令不在白名单 → 直接拒绝; 显式拉黑所有 shell/解释器; log_input/log_output
```

留 io 日志

▼ [L7] AuditLogger 全程事件 → SQLite + JSONL (哈希链/HMAC) , 可外送 SIEM, 攻击者无法只删本机

L4 的人工确认用于平衡安全与效率：低风险只读操作直接放行，高风险或对抗场景下把决策交回人。  
cron-injection-v1 场景的实测结果见《04》§ 5.2。

### 5.3 失败注入点（防御纵深矩阵）

| 攻击姿势                                                    | 在哪一层被挡                                                                        |
|---------------------------------------------------------|-------------------------------------------------------------------------------|
| LLM 幻觉 <code>rm -rf /</code>                            | Guardrail 命中正则 → DENY                                                         |
| <code>svc_restart</code> 喂 <code>sshd; rm -rf /</code>  | <code>build_argv</code> 检测 shell 元字符 → ToolError                              |
| <code>svc_restart</code> 喂 <code>systemd-logind</code>  | Tool 内 <code>_FORBIDDEN_UNITS</code> → ToolError                              |
| 想让 <code>process_list</code> 读 <code>/etc/shadow</code> | 工具无 <code>cat</code> 接口； <code>fs_ls</code> 禁读 <code>/etc/shadow</code>       |
| <code>find -exec rm</code> 绕过 <code>rm</code> 规则        | <code>fs_find</code> 不传 <code>-exec</code>                                    |
| <code>confirm</code> 时被诱导按 yes                          | 仍要过 <code>sudoers</code> 白名单，不在白名单直接失败                                        |
| 进程内核态提权                                                 | <code>rlimit</code> + 干净 <code>env</code> + <code>PATH</code> 白名单 + 非 root 账户 |
| 输出体爆破                                                   | sandbox <code>output_cap</code> 64K/流 + 工具层 6KB 截断                            |
| 死循环消耗 CPU                                               | <code>timeout</code> + <code>SIGTERM/SIGKILL</code> pgrp                      |
| 删本机审计                                                   | SQLite + JSONL, JSONL 外送 SIEM                                                 |

## 6. 配置与权限设计

### 6.1 配置链（`docs/kyagent/architecture.md` 第 3 节）

```
KYAGENT_CONFIG env → configs/default.yaml (YAML + ${VAR:-default} 展开) → Pydantic Config
                                |→ SandboxConfig → ExecutionProxy
                                |→ SafetyConfig → Guardrail.from_config
                                |→ RuleEngine.from_yaml(configs/safety-rules.yaml)
```

`safety-rules.yaml` 与 `default.yaml` 分开，规则库可独立热扩，不需改代码。配置文件清单：`configs/default.yaml`、`deepseek.yaml`、`qwen.yaml`、`openai.yaml`、`intent-rules.yaml`、`safety-rules.yaml`、`rca-playbooks.yaml`、`sudoers.kyagent`。

### 6.2 权限设计（三档 `sudoers`）

| 档位     | 脚本                                    | 含义                                                                                                                                                                        |
|--------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 最小只读基线 | <code>scripts/setup-sudoers.sh</code> | 正式安全模型，只放行必要 root 只读查询（如 <code>dmidecode -s</code> 、 <code>iptables -L</code> 、 <code>nft list ruleset</code> 、 <code>smartctl</code> 、 <code>sudo -l -U kyagent</code> ） |



| 档位   | 脚本                                      | 含义                                                                       |
|------|-----------------------------------------|--------------------------------------------------------------------------|
| 生产预设 | scripts/setup-sudoers-prod.sh --yes     | 最小权限脚本的生产封装，预开常见写操作（日志清理/包管理/进程终止/常见服务 restart-reload），仍按固定命令 + 锚定参数正则放行 |
| 极限测试 | scripts/setup-sudoers-max-test.sh --yes | 仅用于极限测试，写入 NOPASSWD:SETENV: ALL，不代表正常安全模型，禁止生产/正式演示长期保留                  |

三档都写入同一目标 `/etc/sudoers.d/kyagent`，后执行覆盖前一档。写操作经 OS 层包装器（`kyagent-log-clean`、`kyagent-file-delete`、`kyagent-cron-disable`、`kyagent-lock-stale`、`kyagent-unix-socket-stale`、`kyagent-log-dir-perms`、`kyagent-cron-trace`），做 `realpath + O_NOFOLLOW` + `fd` 真实路径越界判定 + `inode` 复检，封死 `..` 越界与符号链接 TOCTOU。

### 6.3 写操作显式授权（opt-in）

默认部署只授予只读查询。需要写能力时显式设置环境变量重跑 `setup-sudoers.sh`：

- `KYAGENT_ENABLE_LOG_CLEAN=1`：`journalctl --vacuum-size/--vacuum-time`、`kyagent-log-clean <绝对路径>`、`kyagent-file-delete <绝对路径>`
- `KYAGENT_ENABLE_PKG_MGMT=1`：`dnf/yum -y install/update/reinstall/clean`、`rpm --rebuilddb`；卸载需 `KYAGENT_PKG_REMOVE_ALLOWLIST`
- `KYAGENT_ENABLE_PROC_KILL=1`：`kill -(TERM|KILL|HUP|INT) <pid>=2+`
- `KYAGENT_SERVICE_ALLOWLIST=...`：业务服务 `restart/reload` 固定命令

## 7. 可扩展性设计

| 扩展点             | 做法                                                                                                                                     |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 新增工具            | 在 <code>kyagent/mcp/tools/</code> 增模块，写 <code>Tool</code> 子类， <code>register()</code> 进 <code>registry</code>                          |
| 新增安全规则          | 在 <code>configs/safety-rules.yaml</code> 加一条（ <code>pattern</code> 或 <code>command+flags+target_in</code> ），无需改代码                      |
| 接其他 LLM         | 实现 <code>LLMBackend.chat()</code> ，统一返回 <code>AssistantMessage</code>                                                                  |
| 接其他 MCP host    | <code>kyagent mcp serve</code> 即标准 <code>stdio</code> ，可被 <code>Claude Desktop</code> / <code>Cursor</code> / <code>Continue</code> 发现 |
| 新增 RCA playbook | 在 <code>configs/rca-playbooks.yaml</code> 增条目                                                                                          |
| 接 SIEM          | 外送审计 JSONL（ <code>/var/log/kyagent/audit.jsonl</code> ）                                                                                |

## 8. 设计与赛题评分项对应

| 评分项             | 设计落点                                  |
|-----------------|---------------------------------------|
| OS 感知与 MCP 插件实现 | 第 3.2 节工具层 + MCP 协议层                  |
| 自然语言交互与准确性      | 第 3.1 节 Agent 核心 + LLM 抽象 + 第 3.7 节前端 |

| 评分项       | 设计落点                                                                                                      |
|-----------|-----------------------------------------------------------------------------------------------------------|
| 安全护栏与风险控制 | 第 3.3/3.4 节 + 第 5/6 节安全模型与权限                                                                              |
| 智能化根因分析能力 | 第 3.5/3.6 节审计 + RCA                                                                                       |
| 创新与实用性    | 受控模板不给 LLM 拼 shell ( § 1 ) 、 7 层防御分层 ( § 5.2 ) 、 人在回路 ( L4 ) ； 零 Rust 国产化、自研 MCP、三档 sudoers、OS 层写包装器、可扩展性 |
| 文档与演示     | B/S Web 控制台 + TUI + CLI 多前端复用同一安全链路                                                                       |